

11.3: AWS Overview



UMD DATA605 - Big Data Systems

11.3: AWS Overview

- **Instructor:** Dr. GP Saggese, gsaggese@umd.edu
- **@References@:**
 - Some basic info in the slides
 - Many tutorials on-line
 - Mastery
 - “Amazon Web Services in Action” 3rd Edition



1 / 33

2 / 33: Amazon Web Services (AWS)

Amazon Web Services (AWS)

- **AWS is a platform offering complete cloud solutions**
 - Computing (e.g., EC2)
 - Storage (e.g., S3)
 - Networking
- **Offers different levels of abstraction**
 - IaaS, PaaS, SaaS
 - From virtual hardware to applications, e.g.,
 - Host websites
 - Run enterprise software
 - Run machine learning applications
- **Control services in different ways**
 - Web interface (console)
 - CLI: `aws` command
 - Language libraries, SDKs (e.g., Python `boto3`)

2 / 33

- **Amazon Web Services (AWS)**
 - *AWS is a platform offering complete cloud solutions*
 - **Computing (e.g., EC2)**: AWS provides Elastic Compute Cloud (EC2), which allows users to rent virtual computers to run their applications. This is a flexible and scalable way to handle computing needs without owning physical hardware.
 - **Storage (e.g., S3)**: Simple Storage Service (S3) is AWS's storage solution, offering scalable and secure storage for data. It's commonly used for storing large amounts of data, such as backups or media files.
 - **Networking**: AWS offers various networking services to connect and manage resources, ensuring secure and efficient data transfer across the cloud.
 - *Offers different levels of abstraction*
 - **IaaS, PaaS, SaaS**: AWS provides Infrastructure as a Service (IaaS), Platform as a Service (PaaS), and Software as a Service (SaaS). These range from basic virtual hardware to complete software solutions, catering to different user needs.
 - **Host websites**: AWS can host websites, providing the necessary infrastructure and tools to manage web traffic and content.
 - **Run enterprise software**: Businesses can run their enterprise applications on AWS, benefiting from its scalability and reliability.
 - **Run machine learning applications**: AWS supports machine learning workloads, offering services like SageMaker to build, train, and deploy models.
 - *Control services in different ways*
 - **Web interface (console)**: Users can manage AWS services through a user-friendly web interface, making it easy to configure and monitor resources.
 - **CLI: `aws` command**: The Command Line Interface (CLI) allows for scripting and

automation of AWS tasks, providing a powerful tool for developers and system administrators.

- **Language libraries, SDKs (e.g., Python boto3):** AWS offers Software Development Kits (SDKs) for various programming languages, such as Python's boto3, enabling developers to integrate AWS services into their applications seamlessly.

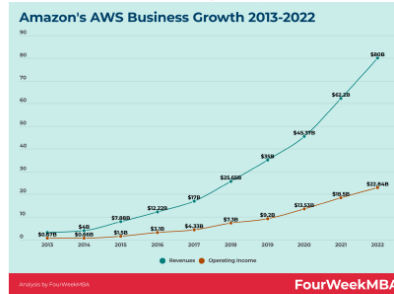
3 / 33: AWS as Business

AWS as Business

- Services charge a pay-per-use pricing model
 - 500 new services and features added annually
- Data centers distributed globally
 - US, Europe, Asia, South America
- Extremely profitable
 - \$91B/year in revenue (2023)
 - Grows 42% year-over-year
 - Controls 30% of the cloud business



Beef Jezos



3 / 33

- **Services charge a pay-per-use pricing model**
 - AWS operates on a *pay-per-use* model, meaning customers only pay for the resources they consume. This flexibility is attractive to businesses of all sizes, allowing them to scale usage according to their needs.
 - With 500 new services and features added annually, AWS continuously evolves to meet diverse customer demands, staying ahead in the competitive cloud market.
- **Data centers distributed globally**
 - AWS has a global presence with data centers in key regions such as the US, Europe, Asia, and South America. This distribution ensures low latency and high availability for users worldwide, enhancing performance and reliability.
- **Extremely profitable**
 - AWS generates \$91 billion in annual revenue as of 2023, showcasing its significant impact on the cloud industry.
 - The business grows at an impressive rate of 42% year-over-year, reflecting its strong market position and customer trust.
 - AWS controls 30% of the cloud business, making it a leader in the industry and a major contributor to Amazon's overall success.

The images humorously depict the transformation of Amazon's business model from selling books

to becoming a dominant force in the cloud computing industry. The growth chart further illustrates AWS's rapid revenue increase over the years.

4 / 33: Types of Cloud Computing

Types of Cloud Computing

- Cloud computing enables a **shared pool of configurable computing resources**
 - E.g., servers, storage, networks, applications, services
 - Accessible from anywhere
 - Convenient
 - On-demand
- Clouds can be:
 - *Public*: open to the general public (e.g., AWS)
 - *Private*: virtualize and share IT infrastructure within one organization (e.g., government)
 - *Hybrid*: mix of public and private clouds

4 / 33

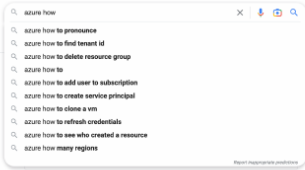
- **Types of Cloud Computing**
 - Cloud computing is a technology that allows users to access a *shared pool of configurable computing resources*. This means that instead of owning and maintaining physical hardware, users can use resources like servers, storage, and applications over the internet.
 - **Examples**: These resources can include things like servers, storage, networks, applications, and services. This flexibility allows businesses to scale their operations without investing in physical infrastructure.
 - **Accessible from anywhere**: One of the key benefits of cloud computing is that it can be accessed from virtually anywhere with an internet connection, making it highly convenient for users.
 - **Convenient**: Users can easily access and manage resources without needing to worry about the underlying hardware.
 - **On-demand**: Resources can be provisioned and released as needed, allowing for efficient use of resources and cost savings.
- **Clouds can be**:
 - **Public**: These are cloud services offered to the general public by companies like Amazon Web Services (AWS). They are typically cost-effective and scalable, but users share the infrastructure with others.
 - **Private**: These clouds are used exclusively by one organization. They offer more control and security, making them ideal for sensitive data or specific regulatory requirements, such as those in government sectors.

- **Hybrid:** This approach combines both public and private clouds, allowing organizations to take advantage of the benefits of both. For example, they might use a private cloud for sensitive data and a public cloud for less critical operations.

5 / 33: AWS vs Google Cloud vs Microsoft Azure

AWS vs Google Cloud vs Microsoft Azure

- **Similarities:**
 - Worldwide infrastructure
 - Provide IaaS (computing, networking, storage)
 - AWS EC2 / Google Compute Engine / Azure VMs
 - AWS S3 / Google Cloud Storage / Azure Blob Storage
 - Pay-as-you-go pricing model
- **Differences:**
 - AWS
 - Market leader, mature, powerful
 - Utilizes open source technologies
 - Azure
 - Offers Microsoft stack in the cloud
 - Google
 - Focuses on cloud-native applications



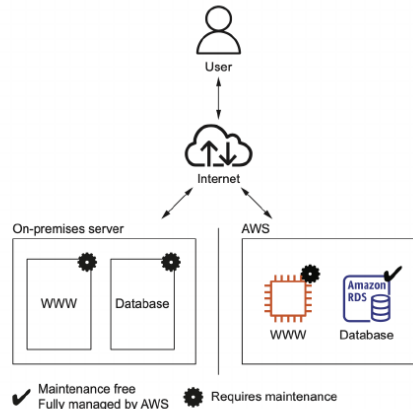
5 / 33

- **Similarities:**
 - All three cloud providers—AWS, Google Cloud, and Microsoft Azure—offer a *global infrastructure*, meaning they have data centers spread across the world. This allows for high availability and low latency.
 - They provide *Infrastructure as a Service (IaaS)*, which includes computing, networking, and storage solutions. For example, AWS offers EC2 for computing, Google Cloud has Compute Engine, and Azure provides Virtual Machines (VMs). Similarly, for storage, AWS has S3, Google Cloud offers Cloud Storage, and Azure provides Blob Storage.
 - They all use a *pay-as-you-go pricing model*, which means you only pay for the resources you use, making it cost-effective and flexible.
- **Differences:**
 - **AWS** is known as the *market leader* in cloud services. It is considered mature and powerful, with a wide range of services and a strong focus on utilizing open-source technologies.
 - **Azure** is particularly strong in offering the *Microsoft stack* in the cloud, making it a preferred choice for businesses already using Microsoft products.
 - **Google Cloud** focuses on *cloud-native applications*, emphasizing modern, scalable, and containerized applications, which is ideal for developers looking to build innovative solutions.

6 / 33: From On-premise to AWS: 1/3

From On-premise to AWS: 1/3

- Aka "Cloud-transformation"
- **Example:** move a medium-sized e-commerce site *from on-premise to the cloud*
- **No-cloud**
 - *Web-server:* handles customer requests
 - *DB:* stores product info and orders
 - *Static content* (e.g., JPEG images)
 - Delivered over CDN
 - Reduces load on services
 - *Dynamic content* (e.g., HTML pages)
 - E.g., products and prices
 - Delivered by web server



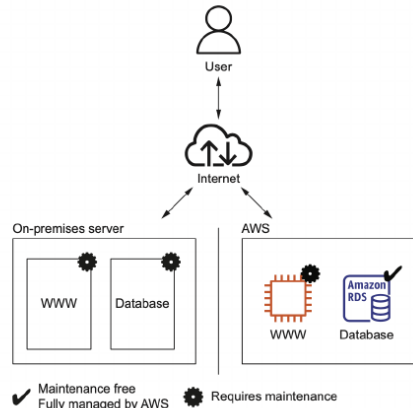
6 / 33

- **From On-premise to AWS: 1/3**
 - This slide introduces the concept of *cloud transformation*, which involves moving services from local servers to cloud-based solutions like AWS.
- **Example: Move a medium-sized e-commerce site from on-premise to the cloud**
 - This example illustrates the transition of a typical e-commerce platform to a cloud environment, highlighting the benefits and changes involved.
- **No-cloud (On-premise)**
 - **Web-server:** Manages customer requests directly on local servers, requiring manual maintenance and scaling.
 - **DB (Database):** Stores critical data such as product information and customer orders, also maintained on-site.
 - **Static content:** Includes items like JPEG images, which are delivered via a Content Delivery Network (CDN) to reduce server load.
 - **Dynamic content:** Consists of HTML pages with real-time data like product details and prices, served by the web server.
- **AWS Transition**
 - The diagram shows a shift from on-premise servers to AWS, where the database can be managed by Amazon RDS, reducing maintenance efforts.
 - AWS offers a more scalable and maintenance-free environment, allowing businesses to focus on growth rather than infrastructure management.

7 / 33: From On-premise to AWS: 2/3

From On-premise to AWS: 2/3

- **Move to cloud**
 - Keep the same architecture
 - Move components to cloud



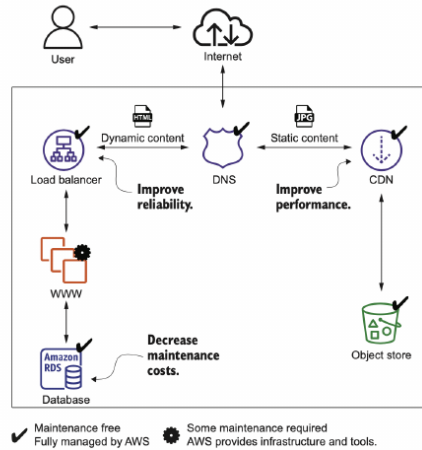
7 / 33

- **Move to cloud**
 - **Keep the same architecture**
 - When transitioning from on-premise to the cloud, one approach is to maintain the existing architecture. This means replicating the current setup in the cloud environment without making significant changes. It allows for a smoother transition as the system's structure remains familiar.
 - **Move components to cloud**
 - This involves transferring specific components of the system to the cloud. For example, the database might be moved to a cloud service like Amazon RDS, which is fully managed by AWS. This reduces the maintenance burden on the organization, as AWS handles updates, backups, and scaling.
- **Diagram Explanation**
 - The diagram illustrates a user accessing services over the internet. On the left, the on-premise setup shows both the web server (WWW) and database requiring maintenance. On the right, the AWS setup shows the web server still requiring maintenance, but the database is now managed by Amazon RDS, which is maintenance-free. This highlights the benefit of using managed services in the cloud to reduce operational overhead.

8 / 33: From On-premise to AWS: 3/3

From On-premise to AWS: 3/3

- **Design for the cloud**
 - DNS
 - Database
 - Object store (S3)
 - Managed solutions
 - Use multiple smaller virtual services with a load balancer to increase reliability



8 / 33

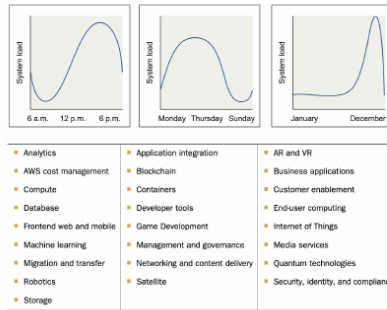
- **Design for the cloud**
 - **DNS:** Domain Name System (DNS) is crucial for translating domain names into IP addresses, enabling users to access services easily. In the cloud, DNS services are optimized for performance and reliability.
 - **Database:** Using cloud-managed databases like Amazon RDS reduces maintenance costs and complexity. AWS handles backups, scaling, and updates, allowing you to focus on application development.
 - **Object store (S3):** Amazon S3 provides scalable storage for objects like images and videos. It's cost-effective and integrates with other AWS services, enhancing data accessibility and durability.
 - **Managed solutions:** AWS offers managed services that handle infrastructure tasks, freeing up resources to focus on innovation and application logic.
 - **Use multiple smaller virtual services with a load balancer:** Distributing traffic across multiple instances with a load balancer increases reliability and availability. This setup ensures that if one instance fails, others can handle the load, maintaining service continuity.
- **Image Explanation**
 - The diagram illustrates a cloud architecture where users interact with services via the internet.
 - **Load balancer:** Enhances reliability by distributing incoming traffic across multiple servers.
 - **DNS:** Improves performance by efficiently routing requests.
 - **CDN:** Content Delivery Network (CDN) accelerates the delivery of static content, improving user experience.

- **Amazon RDS:** A fully managed database service that reduces maintenance overhead.
- **Object store:** Provides scalable storage for static content, integrated with CDN for faster access.

9 / 33: Capacity Scaling

Capacity Scaling

- **No need to plan for capacity**
 - Do not predict capacity
 - Schedule capacity on-the-fly
 - No concern for rack space, switches, power supplies
 - Add more VMs (1 to 1000) and storage (GB to PB) as needed
- **Handle seasonal traffic by scaling up/down**
 - Day vs night
 - Weekday vs weekend
 - Holidays
 - No test system needed when the team is off
- **Worldwide presence**
 - AWS has many data centers
 - Deploy applications close to customers



9 / 33

- **No need to plan for capacity**
 - *Do not predict capacity:* With cloud services, there's no need to estimate future needs. Resources can be adjusted as required.
 - *Schedule capacity on-the-fly:* Resources like virtual machines (VMs) and storage can be allocated dynamically, allowing for immediate scaling.
 - *No concern for rack space, switches, power supplies:* Physical infrastructure concerns are minimized, as cloud providers handle these aspects.
 - *Add more VMs and storage as needed:* Easily scale from a few to thousands of VMs and from gigabytes to petabytes of storage.
- **Handle seasonal traffic by scaling up/down**
 - *Day vs night:* Adjust resources based on daily usage patterns.
 - *Weekday vs weekend:* Scale according to weekly traffic variations.
 - *Holidays:* Increase capacity during peak times like holidays.
 - *No test system needed when the team is off:* Reduce resources when not in use, saving costs.
- **Worldwide presence**
 - *AWS has many data centers:* A global network of data centers ensures low latency and high availability.
 - *Deploy applications close to customers:* Position applications near users to improve performance and user experience.

10 / 33: Pay-per-use

Pay-per-use

– **AWS bill**

- Similar to an electric bill
- Services billed based on usage:
 - Hours of virtual server (rounded up)
 - Used storage in GB (allocated or real)
 - Data traffic in GB or requests
 - ...

Service	January usage	February usage	February charge	Increase
Visits to website	100,000	500,000		
CDN	25 M requests + 25 GB traffic	125 M requests + 125 GB traffic	\$115.00	\$100.00
Static files	50 GB used storage	50 GB used storage	\$1.15	\$0.00
Load balancer	748 hours + 50 GB traffic	748 hours + 250 GB traffic	\$19.07	\$1.83
Web servers	1 virtual machine = 748 hours	4 virtual machines = 2,992 hours	\$200.46	\$150.35
Database (748 hours)	Small virtual machine + 20 GB storage	Large virtual machine + 20 GB storage	\$133.20	\$105.47
DNS	2 M requests	10 M requests	\$4.00	\$3.20
Total cost			\$472.88	\$360.85

– **Free tier**

- Use some AWS services free for 12 months after signing up
- Experiment with services
 - E.g., EC2, S3
- Exceed limits, start paying (without notice)
 - Set an alarm!

10 / 33

– **Pay-per-use Model**

- *AWS Bill*: This is similar to how you pay for utilities like electricity. You are charged based on how much you use the services.
 - **Virtual Server Hours**: You pay for the hours your virtual servers are running, rounded up to the nearest hour.
 - **Storage Usage**: Charges are based on the gigabytes of storage you use, whether it's allocated or actually used.
 - **Data Traffic**: You are billed for the amount of data transferred, measured in gigabytes or by the number of requests.

– **Free Tier**

- AWS offers a *free tier* for new users, allowing them to use certain services without charge for the first 12 months.
- This is a great opportunity to *experiment* with AWS services like EC2 (Elastic Compute Cloud) and S3 (Simple Storage Service).
- Be cautious: if you exceed the free tier limits, you will start incurring charges without any notification. It's wise to set up an *alarm* to monitor usage.

– **Usage and Costs Table**

- The table provides a comparison of service usage and costs between January and February.
- Noticeable increases in usage, such as the jump in website visits and virtual machine hours, lead to higher charges.
- This highlights the importance of monitoring usage to manage costs effectively.

11 / 33: Pay-per-use

Pay-per-use– **Advantages**

- No upfront investments or commitment
- Lower project startup cost
- Easier to divide the system into smaller parts
 - One big server or two smaller ones cost the same
- Affordable fault tolerance/high performance
 - For a scalable workload “buy 1 server for 1000 hours” = “buy 1000 servers for 1 hour”



11 / 33

– **Advantages**

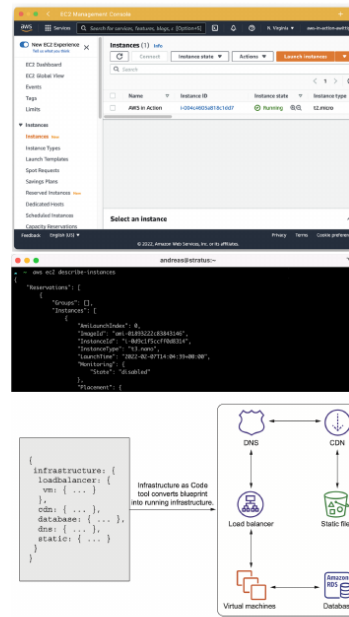
- **No upfront investments or commitment**
 - Pay-per-use models allow businesses to avoid large initial costs. This is particularly beneficial for startups or projects with uncertain futures, as it reduces financial risk.
- **Lower project startup cost**
 - By only paying for what you use, initial expenses are minimized. This makes it easier to launch projects without needing significant capital.
- **Easier to divide the system into smaller parts**
 - Flexibility in resource allocation means you can choose between one large server or multiple smaller ones without cost differences. This allows for tailored solutions that fit specific needs.
- **Affordable fault tolerance/high performance**
 - The model supports scalability. For instance, using one server for 1000 hours is equivalent in cost to using 1000 servers for one hour. This flexibility ensures that systems can handle varying workloads efficiently without overspending.

The image seems to humorously depict strong, influential figures, possibly symbolizing the robustness and flexibility of the pay-per-use model.

12 / 33: Interacting with AWS

Interacting with AWS

- **GUI (Management Console)**
 - Start interacting with services easily
 - Set up cloud infrastructure for development and testing
- **Command-line tool (CLI)**
 - Manage and access AWS services
 - Automate recurring tasks
- **SDKs**
 - Use libraries in any language to interact with AWS
 - Integrate applications with AWS
 - E.g., **boto3** for Python
- **Blueprints**
 - Describe your system with all services and dependencies
 - Describe the system, not how to build it



12 / 33

- **GUI (Management Console)**
 - The AWS Management Console provides a user-friendly graphical interface to interact with AWS services. It is ideal for beginners as it simplifies the process of setting up and managing cloud infrastructure. This tool is particularly useful for development and testing environments where visual feedback is beneficial.
- **Command-line tool (CLI)**
 - The AWS CLI allows users to manage AWS services through commands. It is powerful for automating tasks, making it a preferred choice for developers who need to script and automate repetitive processes. This tool is essential for integrating AWS services into automated workflows.
- **SDKs**
 - AWS SDKs offer libraries for various programming languages, enabling developers to interact with AWS services directly from their applications. This integration is seamless and allows for more complex and customized interactions. For example, **boto3** is the SDK for Python, widely used for scripting and automation.
- **Blueprints**
 - Blueprints describe the architecture of a system, including all services and dependencies, without detailing the construction process. This approach is part of Infrastructure as Code (IaC), where the blueprint is transformed into a running infrastructure, ensuring consistency and repeatability in deployments.

13 / 33: Accounts and Users

Accounts and Users

– **Users**

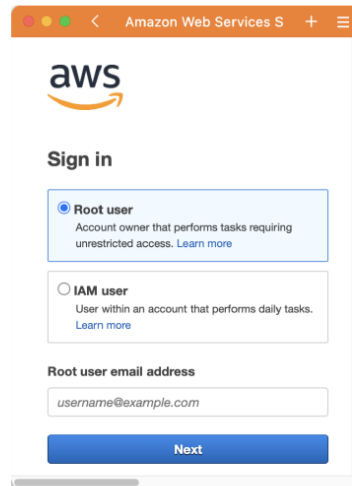
- One root AWS account
- Attach multiple users to an account
 - Different privileges
 - Isolate workloads

– **Be safe**

- Never use root account to develop
- Always use 2FA
- Avoid costly mistakes

– **Key pair**

- Create a key pair to access a virtual server
- Public key (stored in AWS and on virtual servers)
- Private key is your secret
 - Don't lose it; it can't be retrieved



13 / 33

– **Users**

- *One root AWS account:* This is the main account with full access to all AWS services and resources. It's crucial to protect this account as it has unrestricted access.
- *Attach multiple users to an account:* You can create multiple users under the main account, each with specific permissions. This helps in managing access and isolating workloads, ensuring that users only have access to what they need.

– **Be safe**

- *Never use root account to develop:* The root account should be reserved for essential tasks only. Using it for development increases the risk of accidental changes or security breaches.
- *Always use 2FA:* Two-factor authentication adds an extra layer of security, making it harder for unauthorized users to access your account.
- *Avoid costly mistakes:* By limiting the use of the root account and using 2FA, you reduce the risk of making expensive errors.

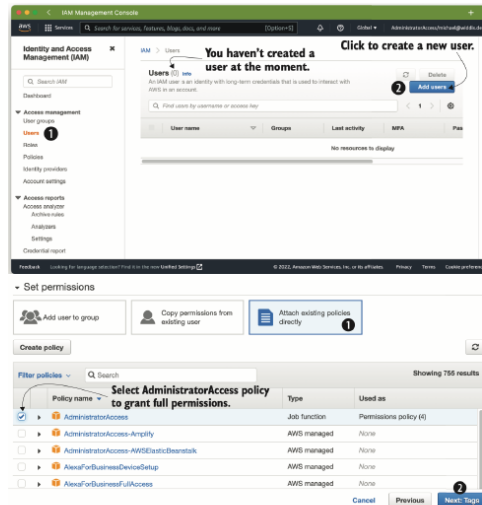
– **Key pair**

- *Create a key pair to access a virtual server:* Key pairs are used for secure access to AWS virtual servers. They consist of a public key and a private key.
- *Public key:* Stored in AWS and on virtual servers, allowing secure communication.
- *Private key is your secret:* It's crucial to keep the private key safe. If lost, it cannot be retrieved, and access to the server will be lost.

14 / 33: Create User Account

Create User Account

- **Create a new user**
 - Do not use the AWS root account for development
 - IAM: Identity and Access Management
 - Access key ID + secret access key
- **Access control**
 - Enable programmatic access
 - Enable console access
 - Limit user actions through policies



14 / 33

- **Create a new user**
 - It's important to avoid using the AWS root account for everyday development tasks. The root account has unrestricted access, which can pose security risks if compromised.
 - **IAM (Identity and Access Management)** is a service that helps manage access to AWS resources securely. It allows you to create and manage AWS users and groups and use permissions to allow and deny their access to AWS resources.
 - When creating a user, you'll receive an *access key ID* and a *secret access key*. These are used for programmatic access to AWS services.
- **Access control**
 - *Programmatic access* enables users to interact with AWS services through APIs, SDKs, or the AWS CLI.
 - *Console access* allows users to log into the AWS Management Console to manage services visually.
 - It's crucial to limit user actions through policies. Policies define permissions and can restrict what actions a user can perform, enhancing security by following the principle of least privilege.

15 / 33: AWS VMs

AWS VMs

- **With virtualization**
 - Multiple VMs run on the same hardware
 - Start and stop VMs on-demand
- **Physical server**
 - Aka “host machine”, “bare metal”
 - Consists of CPUs, memory, networking interfaces, storage
- **Hypervisor**
 - Software + CPU hardware
 - Isolates guests
 - Schedules hardware requests
- **AWS**
 - Used Xen hypervisor (open-source) before
 - Switched to AWS Nitro
 - Hardware-assisted virtualization
 - Performance close to bare

```

graph BT
    G1[Guest 1] --- H[Hypervisor]
    G2[Guest 2] --- H
    G3[Guest 3] --- H
    H --- HM[Host machine]
          
```

15 / 33

- **With virtualization**
 - *Multiple VMs run on the same hardware:* Virtualization allows several virtual machines (VMs) to operate on a single physical server, optimizing resource use and reducing costs.
 - *Start and stop VMs on-demand:* This flexibility means you can easily scale resources up or down based on current needs, enhancing efficiency.
- **Physical server**
 - *Aka “host machine”, “bare metal”:* This is the actual physical hardware that provides the resources for virtualization.
 - *Consists of CPUs, memory, networking interfaces, storage:* These components are essential for running applications and services.
- **Hypervisor**
 - *Software + CPU hardware:* The hypervisor is crucial for managing VMs, acting as a bridge between the physical hardware and the virtual environments.
 - *Isolates guests:* It ensures that each VM operates independently, providing security and stability.
 - *Schedules hardware requests:* Efficiently allocates resources to VMs as needed.
- **AWS**
 - *Used Xen hypervisor (open-source) before:* AWS initially utilized Xen for virtualization.
 - *Switched to AWS Nitro:* AWS transitioned to Nitro for improved performance.
 - *Hardware-assisted virtualization:* Nitro uses specialized hardware to enhance virtualization.
 - *Performance close to bare metal:* Offers near-native performance, making it highly efficient.
- **Virtual servers**

- *Isolated on the same hardware:* Each VM runs independently, ensuring that issues in one do not affect others.

16 / 33: Select Region

Select Region

- E.g., `us-east-1`
 - 21155 Smith Switch Road, Ashburn, VA, USA



- Major internet backbone in Ashburn, VA
 - Equinix
 - Digital Realty
 - Vantage Data Centers
 - H5 Data Centers
 - ...



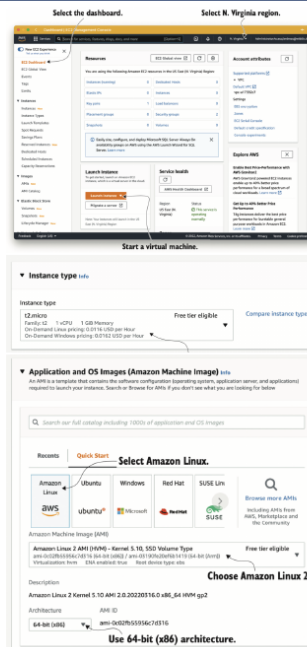
16 / 33

- **Select Region**
 - The slide discusses the importance of selecting a region for data centers, using `us-east-1` as an example. This is a common region identifier used by cloud service providers like AWS.
 - *21155 Smith Switch Road, Ashburn, VA, USA* is highlighted as a key location. Ashburn is known for its dense concentration of data centers.
- **Major Internet Backbone in Ashburn, VA**
 - Ashburn is a critical hub for internet infrastructure, often referred to as “Data Center Alley.”
 - **Equinix, Digital Realty, Vantage Data Centers, H5 Data Centers:** These are some of the major companies operating data centers in the area. They provide essential services for cloud computing, data storage, and internet connectivity.
 - The presence of these companies underscores Ashburn’s role as a major node in the global internet backbone, facilitating vast amounts of data traffic.
- **Images**
 - The images likely depict the physical infrastructure and geographical location of these data centers, emphasizing their scale and strategic placement.
 - The maps provide a visual context of Ashburn’s proximity to major cities like Washington, D.C., highlighting its accessibility and strategic importance.

17 / 33: Starting an EC2 Instance

Starting an EC2 Instance

- **Select OS**
 - Amazon Machine Image (AMI)
 - Contains OS + pre-installed software
 - Saves time installing OS, packages, software
- **Choose instance parameters**
 - E.g., `t2.micro`
 - More info later
- **Configure instance**
 - Network, shutdown behavior, termination protection, monitoring



17 / 33

- **Select OS**
 - *Amazon Machine Image (AMI):* This is a template that includes the operating system and any pre-installed software. It simplifies the setup process by eliminating the need to manually install the OS and software packages.
 - *Saves time:* Using an AMI can significantly reduce the time needed to get your instance up and running, as everything is pre-configured.
- **Choose instance parameters**
 - *Example: `t2.micro`:* This is a specific type of instance that is often used for testing and development due to its low cost and eligibility for the free tier.
 - *More info later:* Details about different instance types and their specifications will be provided later.
- **Configure instance**
 - *Network:* Set up the network settings to ensure your instance can communicate with other resources.
 - *Shutdown behavior:* Decide what happens when the instance is shut down (e.g., stop or terminate).
 - *Termination protection:* Enable this to prevent accidental termination of the instance.
 - *Monitoring:* Set up monitoring to keep track of the instance's performance and health.

These steps are part of launching a virtual machine on AWS using the EC2 service, which allows you to run applications in the cloud efficiently.

18 / 33: AWS Instance Type

AWS Instance Type

- **Instance type** describes the computing power
- **Instance family**
 - **T**: cheap, baseline
 - **M**: general purpose
 - **C**: compute optimized
 - **R**: memory optimized
 - **D**: storage optimized for HDD
 - **I**: storage optimized for SSD
 - **F**: with FPGAs
 - **P, G, CG**: with GPUs
- AWS EC2 list

Instance	vCPU*	CPU Credits / hour	Mem (GiB)	Storage	Network Performance
t2.nano	1	3	0.5	EBS-Only	Low
t2.micro	1	6	1	EBS-Only	Low to Moderate
t2.small	1	12	2	EBS-Only	Low to Moderate
t2.medium	2	24	4	EBS-Only	Low to Moderate
t2.large	2	36	8	EBS-Only	Low to Moderate
t2.xlarge	4	54	16	EBS-Only	Moderate
t2.2xlarge	8	81	32	EBS-Only	Moderate

- E.g., **t2.micro**
 - **t**: instance family (small, cheap)
 - **2**: generation (second)
 - **micro**: size (1 vCPU, 1GB memory)
 - 0.013 USD/hr
- E.g., **m4.large**
 - **m**: general purpose
 - **large**: size (2 vCPUs, 8GB memory)
 - 0.10 USD/hr

18 / 33

- **AWS Instance Type**: This refers to the specific configuration of virtual hardware that AWS provides. It determines the computing power available for your tasks.
- **Instance Family**: AWS categorizes instances into families based on their primary use case:
 - **T**: *Cheap, baseline performance* instances suitable for low-cost applications.
 - **M**: *General purpose* instances that balance compute, memory, and networking resources.
 - **C**: *Compute optimized* instances designed for compute-intensive applications.
 - **R**: *Memory optimized* instances for memory-intensive tasks.
 - **D**: *Storage optimized for HDD* for applications requiring high disk throughput.
 - **I**: *Storage optimized for SSD* for high random I/O performance.
 - **F**: Instances with *FPGAs* for hardware acceleration.
 - **P, G, CG**: Instances with *GPUs* for graphics or compute-intensive tasks.
- **AWS EC2 List**: A comprehensive list of available instance types can be found on the AWS EC2 instance types page.
- **Example Instances**:
 - **t2.micro**:
 - *Family*: **t** (small, cheap)
 - *Generation*: **2** (second)
 - *Size*: **micro** (1 vCPU, 1GB memory)
 - *Cost*: \$0.013 per hour
 - **m4.large**:
 - *Family*: **m** (general purpose)
 - *Size*: **large** (2 vCPUs, 8GB memory)

- *Cost:* \$0.10 per hour

These examples illustrate how AWS instances are named and priced, helping users choose the right configuration for their needs.

19 / 33: AWS Instance Type

AWS Instance Type

- Pricing on the AWS website can be unclear due to various factors:
 - Burst mode
 - vCPUs
 - Multi-tenancy
 - On-demand vs. spot vs. reserved vs. pre-paid
- 642 types of EC2 machines (as of 2023)
 - Cheapest: \$37 USD/year
 - Most expensive: \$1.91M USD/year (500 CPUs and 24TB of memory)
- Alternative sites tracking prices

19 / 33

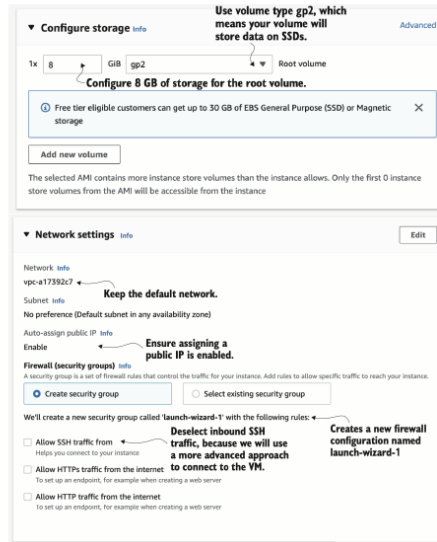
- **AWS Instance Type**
 - When looking at AWS pricing, it can be confusing because there are many factors that affect the cost.
 - **Burst mode:** Some instances can temporarily increase their performance, which can affect pricing.
 - **vCPUs:** This stands for virtual CPUs, which are the processing power of the instance. More vCPUs generally mean higher costs.
 - **Multi-tenancy:** This refers to whether the instance is shared with other users or dedicated to one user, impacting the price.
 - **On-demand vs. spot vs. reserved vs. pre-paid:** These are different pricing models. On-demand is pay-as-you-go, spot instances are cheaper but can be interrupted, reserved instances are paid upfront for a discount, and pre-paid options can offer savings.
- **642 types of EC2 machines (as of 2023)**
 - AWS offers a wide range of EC2 instances to fit different needs and budgets.
 - The **cheapest instance** costs about \$37 USD per year, which is suitable for very light workloads.
 - The **most expensive instance** can cost up to \$1.91 million USD per year, offering massive computing power with 500 CPUs and 24TB of memory, ideal for large-scale enterprise applications.

- **Alternative sites tracking prices**
 - There are websites like Vantage that help track and compare AWS instance prices, which can be useful for finding the best deals and understanding pricing trends.

20 / 33: Starting an EC2 Instance

Starting an EC2 Instance

- Add storage
 - Volume size
 - Volume type (SSD or magnetic HDD)
- Tag
- Configure firewall
 - How to access using SSH
 - Select key pair
- How to monitor the instance
 - E.g., CloudWatch



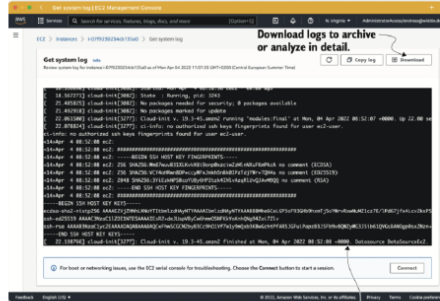
20 / 33

- **Add storage**
 - *Volume size*: This refers to the amount of storage you allocate to your EC2 instance. In the example, 8 GiB is configured for the root volume.
 - *Volume type*: You can choose between SSD (Solid State Drive) and magnetic HDD (Hard Disk Drive). SSDs, like the gp2 type shown, offer faster performance.
- **Tag**
 - Tags are labels that help you organize and manage your AWS resources. They can include information like the environment (e.g., development, production) or the project name.
- **Configure firewall**
 - *How to access using SSH*: Secure Shell (SSH) is a protocol used to securely connect to your instance. Ensure SSH access is configured correctly.
 - *Select key pair*: A key pair is used for SSH access. You must select or create a key pair to connect securely to your instance.
- **How to monitor the instance**
 - *E.g., CloudWatch*: AWS CloudWatch is a monitoring service that provides data and actionable insights to monitor your applications, respond to system-wide performance changes, and optimize resource utilization.

21 / 33: Starting an EC2 Instance

Starting an EC2 Instance

- Start instance
- Find public IP



- Connect to the machine

```
> ssh -i $PATH/mykey.pem ubuntu@$PUBLIC_IP
> cat /proc/cpuinfo
> free -m
> sudo apt-get update
> sudo apt-get install ...
```

21 / 33

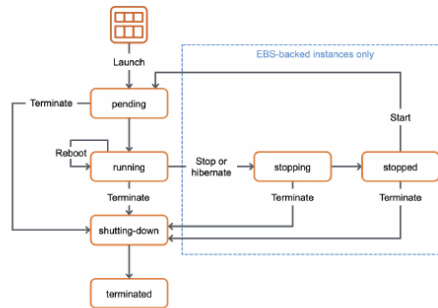
- **Start instance**
 - To begin using an EC2 instance, you first need to start it from the AWS Management Console. This involves selecting the instance you want to run and clicking the “Start” button. This action boots up the virtual server.
- **Find public IP**
 - Once the instance is running, you need to locate its public IP address. This IP is essential for connecting to the instance remotely. You can find it in the instance details within the AWS console.
- **Connect to the machine**
 - Use SSH to connect to your EC2 instance. The command provided uses a private key (mykey.pem) and the public IP address of the instance.
 - ```
> ssh -i $PATH/mykey.pem ubuntu@$PUBLIC_IP
```
  - After connecting, you can check system information using commands like `cat /proc/cpuinfo` for CPU details and `free -m` for memory usage.
  - Update the system and install necessary packages using:
 

```
> sudo apt-get update
> sudo apt-get install ...
```
- **Download logs**
  - The image shows the EC2 Management Console where you can download system logs. These logs are useful for troubleshooting and analyzing the instance’s behavior. You can copy or download them for further inspection.

## 22 / 33: States of a VM

## States of a VM

- **Start**
  - Start stopped VM
- **Stop**
  - Stopped VM not billed
  - Network HDD persists and incurs charges
  - Local disk does not persist
  - VM can restart on a different host (different IP)
- **Reboot**
  - Network HDD persists
  - Local disk does not persist
  - Software remains installed
  - VM restarts on a different host
- **Terminate**
  - Delete: cannot restart
  - Network HDD persists
  - Local disk wiped out



22 / 33

- **States of a VM**
- **\*\*@Start@\*\***
  - This action initiates a stopped virtual machine (VM), bringing it back to an active state.
- **\*\*@Stop@\*\***
  - When a VM is stopped, it is not billed for compute resources.
  - The network-attached hard disk drive (HDD) continues to exist and incurs storage charges.
  - The local disk storage does not persist, meaning any data stored locally is lost.
  - Upon restarting, the VM may be assigned to a different host, resulting in a different IP address.
- **\*\*@Reboot@\*\***
  - The network HDD remains intact, preserving data.
  - Local disk storage is not retained, leading to data loss on local drives.
  - Installed software remains unaffected and continues to exist on the VM.
  - The VM may restart on a different host, similar to the stop/start process.
- **\*\*@Terminate@\*\***
  - This action deletes the VM, making it impossible to restart.
  - The network HDD persists, allowing data recovery or reuse.
  - Local disk storage is completely wiped out, erasing all locally stored data.

The diagram illustrates the lifecycle of a VM, showing transitions between states like pending, running, stopping, and terminated. It highlights how actions like start, stop, reboot, and terminate affect the VM's state and data persistence.

## 23 / 33: Moving / Upgrading EC2 Instances

### Moving / Upgrading EC2 Instances

- **Scale up / down**
  - Increase VM size for more computing power
  - Stop VM
  - Change instance type (e.g., `m3.large`)
  - Start VM
  - IP addresses change
- **AWS regions**
  - Regions are collections of data centers
  - Regions are independent
  - No automatic data transfer across regions
  - Some services (e.g., IAM, CDN, DNS) are global
- **Why move across AWS regions**
  - Proximity to users
  - Compliance
    - Data storage and processing permissions
  - Service availability
    - Some services unavailable in certain regions
  - Redundancy
  - Costs
    - Vary by region

23 / 33

- **Moving / Upgrading EC2 Instances**
- *Scale up / down*
  - When you need more computing power, you can **increase the size of your virtual machine (VM)**. This means choosing a larger instance type that offers more CPU, memory, or other resources.
  - To change the instance type, you must first **stop the VM**. This is necessary because the instance type cannot be changed while the VM is running.
  - After stopping the VM, you can **change the instance type** to something like `m3.large`, which is a specific configuration of resources.
  - Once the instance type is changed, you can **start the VM** again to resume operations.
  - Be aware that **IP addresses will change** when you stop and start the VM, which might affect how you connect to it.
- **AWS regions**
  - AWS regions are essentially **collections of data centers** located in different parts of the world.
  - Each region operates **independently**, meaning they don't automatically share data with each other.
  - There is **no automatic data transfer across regions**, so if you need data in multiple

regions, you must handle the transfer yourself.

- Some AWS services, like **IAM (Identity and Access Management)**, **CDN (Content Delivery Network)**, and **DNS (Domain Name System)**, are considered global and are not confined to a single region.
- **Why move across AWS regions**
  - **Proximity to users** is a key reason to choose a specific region. Hosting your services closer to your users can reduce latency and improve performance.
  - **Compliance** is another factor, as different regions have different rules about data storage and processing. You may need to store data in a specific region to comply with local laws.
  - **Service availability** can vary by region. Some AWS services might not be available in every region, so you might need to move to a region where the services you need are offered.
  - **Redundancy** is important for disaster recovery. By using multiple regions, you can ensure that your services remain available even if one region experiences an outage.
  - **Costs** can differ between regions, so you might choose a region based on pricing to optimize your expenses.

## 24 / 33: Optimizing Costs

### Optimizing Costs

---

- **On-demand instances**
  - Maximum flexibility, no restrictions
  - Start and stop VMs anytime
  - Pay by hour
- **EC2 / Compute saving plans**
  - 1 yr vs 3 yrs
  - Commit to a certain number of hours
  - Payment options: all, partial, no upfront
  - Discount: up to 3x cheaper than on-demand
  - Useful for dev servers
- **Capacity reservation**
  - Access machines even in peak hours
- **Spot instances**
  - Bid for unused capacity
  - Price based on supply/demand
  - Discount: up to 10x cheaper than on-demand
  - Useful for asynchronous tasks

24 / 33

- **Optimizing Costs**
  - **On-demand instances**
    - These instances offer *maximum flexibility* because there are no long-term commitments or restrictions. You can start and stop virtual machines (VMs) whenever you need, which is great for unpredictable workloads or short-term projects. The cost



is straightforward as you pay by the hour for the time you use the instances. This option is ideal if you need quick access to computing resources without any upfront commitment.

- **EC2 / Compute saving plans**
  - With these plans, you can choose between a 1-year or 3-year commitment to a certain number of computing hours. This commitment allows you to receive significant discounts, making it up to 3 times cheaper than on-demand instances. You have flexible payment options: pay everything upfront, partially, or nothing upfront. These plans are particularly useful for development servers where you have a predictable workload and can commit to long-term usage.
- **Capacity reservation**
  - This option ensures that you have access to the computing resources you need, even during peak usage times. It's like reserving a spot in advance, so you don't have to worry about availability when demand is high. This is crucial for applications that require guaranteed performance and availability.
- **Spot instances**
  - Spot instances allow you to bid for unused computing capacity at a significantly reduced cost, sometimes up to 10 times cheaper than on-demand instances. The price fluctuates based on supply and demand, so it's ideal for tasks that are flexible with timing, such as asynchronous or batch processing tasks. This option is great for cost savings if your applications can handle interruptions and are not time-sensitive.

## 25 / 33: Low-Cost Processing

### Low-Cost Processing

---

- Lots of batch jobs are not critical / run on a schedule
  - Analyze data daily
  - Generate reports from a database
- 1. **Allocate machines on demand**
  - AWS bills VMs per minute
  - Pay only when running jobs
- 2. **AWS Batch**
  - Offers spare capacity at a discount
  - Runs when capacity is available, saving 50%
- 3. **AWS Lambda**
  - Serverless

25 / 33

- **Low-Cost Processing**
  - Many batch jobs are not time-sensitive and can be scheduled to run at specific times.

This means they don't need to be processed immediately and can be planned to optimize costs.

- *Analyze data daily*: This involves processing data once a day, which is often sufficient for many business needs. It allows for the accumulation of data over a 24-hour period before analysis.
- *Generate reports from a database*: Reports are typically generated periodically, such as daily or weekly, and do not require real-time processing. This makes them ideal candidates for scheduled batch processing.

### 1. Allocate machines on demand

- Using cloud services like AWS, you can allocate virtual machines (VMs) only when you need them. AWS charges for VMs by the minute, so you only pay for the time the machines are actually running jobs. This is cost-effective for non-critical batch jobs that don't need constant processing power.

### 2. AWS Batch

- AWS Batch is a service that allows you to run batch computing workloads using AWS's spare capacity. This spare capacity is offered at a discounted rate, which can save you up to 50% compared to regular pricing. It runs jobs when there is available capacity, making it a cost-efficient option for non-urgent tasks.

### 3. AWS Lambda

- AWS Lambda is a serverless computing service that lets you run code without provisioning or managing servers. It automatically scales with the size of the workload and you only pay for the compute time you consume. This is ideal for small, event-driven tasks that can be executed quickly and efficiently.

## 26 / 33: Programming the Infrastructure

### Programming the Infrastructure

- On AWS everything can be **controlled via an API**

- Start a VM
- Create 1TB of storage
- Start a Hadoop cluster

- Use:

- AWS GUI console
- HTTP requests to API
- CLI (Command Line Interface)
  - Call AWS API from terminal
- SDK (Software Development Kit)
  - Call AWS API from code
- CloudFormation: templates describe infrastructure state, translated into API calls

1. All teams will henceforth expose their data and functionality through service interfaces.
2. Teams must communicate with each other through these interfaces.
3. There will be no other form of interprocess communication allowed: no direct linking, no direct reads of another team's data store, no shared-memory model, no back-doors whatsoever. The only communication allowed is via service interface calls over the network.
4. It doesn't matter what technology they use. HTTP, Corba, Pubsub, custom protocols – doesn't matter.
5. All service interfaces, without exception, must be designed from the ground up to be externalizable. That is to say, the team must plan and design to be able to expose the interface to developers in the outside world. No exceptions.
6. Anyone who doesn't do this will be fired.
7. Thank you; have a nice day!

- **Jeff Bezos 2002 API mandate**
  - An email worth \$100B/yr

- **On AWS everything can be *controlled via an API***
  - AWS allows users to manage resources programmatically. This means you can automate tasks like starting a virtual machine (VM), creating storage, or launching a Hadoop cluster using API calls.
- **Use:**
  - **AWS GUI console:** A web-based interface for managing AWS services.
  - **HTTP requests to API:** Directly interact with AWS services using HTTP requests.
  - **CLI (Command Line Interface):** Execute AWS API commands from the terminal, offering a scriptable way to manage resources.
  - **SDK (Software Development Kit):** Integrate AWS services into your applications by calling APIs from your code.
  - **CloudFormation:** Use templates to define infrastructure, which are then translated into API calls to create and manage AWS resources.
- **Jeff Bezos 2002 API mandate**
  - This mandate emphasized the importance of service interfaces for communication between teams. It required all teams to expose their data and functionality through APIs, ensuring no direct access to data stores or shared memory. This approach facilitated scalability and externalization, contributing significantly to AWS's success.

## 27 / 33: Infrastructure-as-Code

### Infrastructure-as-Code

---

- Use high-level **programming language to control IT systems**
- Apply **software development principles to infrastructure**
  - Code repository
  - Automated tests
  - Continuous integration
- **DevOps (SRE in Google parlance)**
  - Mix developers and operations in the same team
  - Use software to bring development and operations closer
  - Switch roles to experience each other's challenges
    - Devs → responsible for operational tasks (e.g., on-call)
    - Ops → involved in software development, making systems easier to operate
  - Foster communication and collaboration

27 / 33

- **Infrastructure-as-Code**
  - The idea here is to manage and provision IT infrastructure using code, just like you would with software applications. This means you can use a *high-level programming language* to automate the setup and management of servers, networks, and other infrastructure components. This approach makes infrastructure management more efficient and less

error-prone.

- **Apply software development principles to infrastructure**
  - By treating infrastructure as code, you can store it in a *code repository*, just like application code. This allows for version control and collaboration.
  - You can also run *automated tests* to ensure that changes to the infrastructure code don't break anything.
  - *Continuous integration* means that changes to the infrastructure code are automatically tested and integrated, ensuring that the infrastructure is always in a deployable state.
- **DevOps (SRE in Google parlance)**
  - DevOps is about bringing together developers and operations teams to work more closely. This collaboration helps in building and maintaining systems that are both reliable and scalable.
  - By using software tools and practices, DevOps aims to bridge the gap between development and operations, making the process more seamless.
  - An important aspect is role-switching, where developers take on operational tasks like being on-call, and operations staff get involved in development. This helps each team understand the other's challenges and fosters better *communication and collaboration*.

## 28 / 33: Infrastructure-as-Code: Advantages

### Infrastructure-as-Code: Advantages

- **Save time**
  - Reuse scripts or blueprints
  - Automate tasks
  - Copy-paste vs. click-click-click
- **Fewer mistakes**
  - Push-button flow
- **Consistency of actions**
  - Multiple deployments per day
- **Deployment pipeline**
  - Commit changes to source code
  - Build application from source
  - Automatic tests (e.g., integration tests)
  - Build testing environment
  - Run acceptance tests in isolation
  - Propagate changes to production
- **Scripts as detailed documentation**
  - Explains what and how, not why



28 / 33

- **Save time**
  - *Reuse scripts or blueprints*: By using pre-written scripts, you can quickly set up infrastructure without starting from scratch each time.
  - *Automate tasks*: Automation reduces manual intervention, speeding up processes and reducing human error.
  - *Copy-paste vs. click-click-click*: Using code to manage infrastructure is faster and more

efficient than manually configuring settings through a user interface.

- **Fewer mistakes**
  - *Push-button flow*: Automating deployments with scripts minimizes the chance of human error, as the same process is repeated consistently.
- **Consistency of actions**
  - *Multiple deployments per day*: Infrastructure-as-Code (IaC) allows for frequent and reliable deployments, ensuring that environments remain consistent.
- **Deployment pipeline**
  - *Commit changes to source code*: Changes are tracked and managed through version control, ensuring transparency and traceability.
  - *Build application from source*: Automates the building process, ensuring that the latest code is always used.
  - *Automatic tests (e.g., integration tests)*: Tests are run automatically to catch issues early in the deployment process.
  - *Build testing environment*: Creates a separate environment for testing, ensuring that tests do not affect production.
  - *Run acceptance tests in isolation*: Ensures that new changes meet requirements before going live.
  - *Propagate changes to production*: Once tests pass, changes are automatically deployed to production, reducing downtime.
- **Scripts as detailed documentation**
  - *Explains what and how, not why*: Scripts serve as a record of what actions are taken and how they are executed, providing clear documentation of the infrastructure setup.

## 29 / 33: AWS Command Line Interface (CLI)

### AWS Command Line Interface (CLI)

---

- Provides a **unified interface** to all AWS services
  - Output is in JSON format
- > apt-get install awscli
- **Authenticate**
  - > aws configure
    - AWS access key ID
      - E.g., `AKIAIR...D7ZCA`
    - AWS secret access key
      - E.g., `SSKIng7jkA...enqBj7`
    - Default region name
      - E.g., `us-east-1`
- **Execute command**
  - > aws <service> <action> --key value

- **AWS Command Line Interface (CLI)**
  - The AWS CLI is a tool that allows you to interact with AWS services using commands in your command-line shell. This is useful for automating tasks and managing AWS resources without needing to use the AWS Management Console.
  - **Provides a *unified interface* to all AWS services**
    - This means you can use the same tool to manage different AWS services, making it easier to automate workflows and integrate AWS into your existing processes.
    - The output from AWS CLI commands is typically in JSON format, which is a lightweight data-interchange format that's easy for both humans and machines to read and write.
    - To install the AWS CLI on a Linux system, you can use the command: `apt-get install awscli`. This command uses the package manager to download and install the AWS CLI tool.
- **Authenticate**
  - Before you can use the AWS CLI, you need to authenticate it with your AWS account credentials. This is done using the `aws configure` command.
  - **AWS access key ID**
    - This is a unique identifier associated with your AWS account. It acts like a username for programmatic access.
  - **AWS secret access key**
    - This is a secret key that acts like a password. It should be kept confidential to ensure the security of your AWS resources.
  - **Default region name**
    - AWS services are hosted in different regions around the world. By setting a default region, you specify where your commands will be executed unless you specify otherwise.
- **Execute command**
  - Once authenticated, you can execute commands using the AWS CLI. The general format is: `aws <service> <action> --key value`.
  - Here, `<service>` refers to the AWS service you want to interact with (e.g., EC2, S3), and `<action>` is the specific operation you want to perform (e.g., start, stop, list).
  - The `--key value` part allows you to specify additional options or parameters for the command, tailoring it to your specific needs.

## 30 / 33: Software Development Kit (SDK)

## Software Development Kit (SDK)

- **SDK is a library for calling AWS API** from your programming language
  - E.g., Python, Go, C++, JavaScript
- **Pros:** it handles
  - Authentication
  - Retry on error
  - HTTPS communication
  - XML / JSON de-/serialization
- **Cons**
  - Imperative approach
  - Dealing with dependencies

```
import boto3

Get the service resource.
dynamodb = boto3.resource('dynamodb')

Create the DynamoDB table.
table = dynamodb.create_table(
 TableName='users',
 KeySchema=[
 {
 'AttributeName': 'username',
 'KeyType': 'HASH'
 },
 {
 'AttributeName': 'last_name',
 'KeyType': 'RANGE'
 }
],
 AttributeDefinitions=[
 {
 'AttributeName': 'username',
 'AttributeType': 'S'
 },
 {
 'AttributeName': 'last_name',
 'AttributeType': 'S'
 }
],
 ProvisionedThroughput={
 'ReadCapacityUnits': 5,
 'WriteCapacityUnits': 5
 }
)

Wait until the table exists.
table.wait_until_exists()

Print out some data about the table.
print(table.item_count)
```

30 / 33

- **SDK Overview**
  - An SDK, or Software Development Kit, is a collection of tools and libraries that allows developers to interact with AWS services using their preferred programming languages, such as Python, Go, C++, and JavaScript.
  - It simplifies the process of making API calls to AWS services, enabling developers to integrate AWS functionalities into their applications more easily.
- **Pros of Using SDKs**
  - **Authentication:** SDKs manage the authentication process, ensuring secure access to AWS services.
  - **Retry on Error:** They automatically handle retries in case of errors, improving the reliability of applications.
  - **HTTPS Communication:** SDKs ensure secure data transmission over HTTPS.
  - **XML/JSON De-/Serialization:** They handle the conversion of data formats, making it easier to work with AWS services that use XML or JSON.
- **Cons of Using SDKs**
  - **Imperative Approach:** SDKs often require an imperative programming style, which may not be as intuitive for some developers.
  - **Dealing with Dependencies:** Managing dependencies can be challenging, especially when integrating multiple AWS services.
- **Code Example Explanation**
  - The code snippet demonstrates how to use the Boto3 SDK in Python to interact with AWS DynamoDB.
  - It shows how to create a DynamoDB table named 'users' with specific key schemas and attribute definitions.

- The code waits for the table to be created and then prints the item count, illustrating basic operations with DynamoDB using the SDK.

## 31 / 33: AWS CloudFormation

### AWS CloudFormation

---

- **Use templates to describe infrastructure**
  - E.g., JSON, YAML
- **Declarative vs imperative approach**
  - Declare the system, not the steps to build it
- **AWS Stacks** processes CloudFormation templates
- **Pros**
  - Consistent infrastructure description
  - Handles dependencies
  - Customizable
    - Inject parameters to customize templates
  - Testable
    - Create infrastructure from a template, test, shut down
  - Updatable
    - Update template, Stacks apply changes automatically
  - Serves as documentation
    - Use as code in source control

31 / 33

- **AWS CloudFormation**
  - *Use templates to describe infrastructure*
    - AWS CloudFormation allows you to define your cloud resources using templates. These templates can be written in JSON or YAML, which are both human-readable formats. This means you can easily describe what your infrastructure should look like without manually setting up each component.
  - *Declarative vs imperative approach*
    - With CloudFormation, you use a declarative approach. This means you specify what the final system should look like, rather than detailing each step to build it. It's like saying "I want a house with three bedrooms" instead of "first lay the foundation, then build the walls."
  - *AWS Stacks processes CloudFormation templates*
    - When you create a CloudFormation template, AWS uses something called a "Stack" to process and manage the resources defined in your template. This helps automate the setup and management of your infrastructure.
- **Pros**
  - **Consistent infrastructure description**
    - Using templates ensures that your infrastructure is described consistently every time you deploy it, reducing errors.
  - **Handles dependencies**
    - CloudFormation automatically manages dependencies between resources, so you



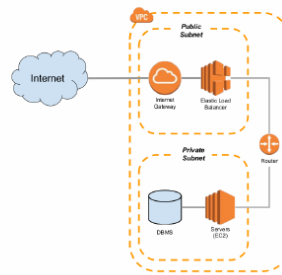
don't have to worry about the order of creation.

- **Customizable**
  - You can inject parameters into your templates to customize them for different environments or use cases.
- **Testable**
  - You can create infrastructure from a template, test it, and then shut it down, which is great for development and testing purposes.
- **Updatable**
  - If you need to make changes, you can update the template, and the Stack will apply those changes automatically.
- **Serves as documentation**
  - Since templates are written as code, they can be stored in source control systems, serving as documentation for your infrastructure.

## 32 / 33: Securing Your System

### Securing Your System

- **Always install software updates**
  - Fix security vulnerabilities promptly
- **Restrict access to AWS account**
  - Use separate AWS accounts for individuals and scripts
  - Apply “least privilege”: grant only necessary permissions
- **Restrict network traffic**
  - Open essential ports
    - E.g., 80 for HTTP, 443 for HTTPS
  - Close others
  - Encrypt traffic and data
- **Create a private network**
  - Use subnets not reachable from the Internet



32 / 33

- **Always install software updates**
  - Keeping your software up-to-date is crucial because updates often include patches for security vulnerabilities. These vulnerabilities can be exploited by attackers, so prompt installation helps protect your system.
- **Restrict access to AWS account**
  - It's important to manage access to your AWS account carefully. By using separate accounts for individuals and scripts, you can better control who has access to what. The principle of “least privilege” means giving users only the permissions they need to perform their tasks, reducing the risk of accidental or malicious actions.
- **Restrict network traffic**

- Limiting network traffic to only essential ports, such as 80 for HTTP and 443 for HTTPS, helps minimize exposure to potential attacks. Closing unnecessary ports and encrypting both traffic and data further enhances security by protecting information from being intercepted or tampered with.
- **Create a private network**
  - Setting up a private network involves using subnets that are not directly accessible from the Internet. This adds an extra layer of security by isolating sensitive resources, such as databases and internal servers, from external threats. The diagram illustrates a typical setup with a public subnet for internet-facing components and a private subnet for internal resources.

## 33 / 33: AWS Shared-responsibility Principle

### AWS Shared-responsibility Principle

---

- **AWS is responsible for:**
  - Protecting the network by monitoring Internet access
  - Preventing DDoS attacks
  - Ensuring the physical security of data centers
  - Decommissioning storage devices after end of life
- **You are responsible for:**
  - Restricting access using IAM
  - Encrypting network traffic (e.g., HTTPS)
  - Configuring firewalls and VPNs
  - Encrypting data
  - Updating the OS and software

33 / 33

- **AWS Shared-responsibility Principle**
  - This principle is a key concept when using AWS services. It defines the division of security and compliance responsibilities between AWS and the customer. Understanding this division is crucial for maintaining a secure cloud environment.
- **AWS is responsible for:**
  - *Protecting the network by monitoring Internet access:* AWS ensures that the infrastructure is secure by monitoring and controlling access to its network.
  - *Preventing DDoS attacks:* AWS has measures in place to protect against Distributed Denial of Service attacks, which can overwhelm systems and cause downtime.
  - *Ensuring the physical security of data centers:* AWS takes care of the physical security of its data centers, ensuring that only authorized personnel have access.
  - *Decommissioning storage devices after end of life:* AWS securely handles the disposal of storage devices to prevent data leaks.

- **You are responsible for:**
  - *Restricting access using IAM:* You must manage who can access your AWS resources by setting up Identity and Access Management (IAM) policies.
  - *Encrypting network traffic (e.g., HTTPS):* It's your job to ensure that data transmitted over the network is encrypted to protect it from interception.
  - *Configuring firewalls and VPNs:* You need to set up firewalls and Virtual Private Networks (VPNs) to control and secure traffic to and from your AWS resources.
  - *Encrypting data:* You should encrypt your data both at rest and in transit to safeguard it from unauthorized access.
  - *Updating the OS and software:* Keeping your operating systems and applications up to date is your responsibility to protect against vulnerabilities.